

实验一：图像滤波

一、实验目的

1. 掌握数字图像处理中卷积运算的基本原理和实现方法
2. 理解 Sobel 算子的工作原理及其在边缘检测中的应用
3. 学会使用自定义卷积核对图像进行滤波处理
4. 掌握图像颜色直方图的计算方法和可视化技术
5. 了解灰度共生矩阵（GLCM）及其在纹理特征提取中的应用

二、实验原理

2.1 卷积运算

卷积是图像处理中最基本的操作之一，其数学表达式为：

$$g(x, y) = \sum_{i=-k}^k \sum_{j=-k}^k f(x+i, y+j) \cdot h(i, j)$$

其中， $f(x, y)$ 为输入图像， $h(i, j)$ 为卷积核， $g(x, y)$ 为输出图像。卷积核在图像上滑动，计算每个位置的加权求和值。

2.2 Sobel 算子

水平方向（X 方向）：

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

垂直方向（Y 方向）：

$$S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

梯度幅值计算公式：

$$G = \sqrt{G_x^2 + G_y^2}$$

2.3 颜色直方图

颜色直方图是图像中像素强度分布的统计表示，它统计每个灰度级（0-255）出现的频次。对于 RGB 彩色图像，需要分别计算三个通道的直方图。

2.4 灰度共生矩阵（GLCM）

灰度共生矩阵是描述图像纹理特征的重要工具，它统计了图像中具有特定空间关系的像素对的灰度组合分布。从 GLCM 中可以提取多种纹理特征，如：

- 对比度（Contrast）：反映图像的清晰度和纹理沟纹深浅
- 相关性（Correlation）：反映图像纹理的一致性
- 能量（Energy）：反映图像灰度分布的均匀程度
- 同质性（Homogeneity）：反映图像纹理的同质性
- 熵（Entropy）：反映图像的随机性和复杂程度

三、实验方法

3.1 实验环境

- 操作系统：Windows 10/11
- Python 版本：3.8-3.11
- 主要依赖库及版本：
 - NumPy 1.24.3（数值计算和矩阵运算）
 - Pillow 10.0.0（图像读取和保存）
 - Matplotlib 3.7.2（数据可视化）

3.2 算法实现

3.2.1 2D 卷积实现

```
# 实现2D卷积操作
def convolution_2d(image, kernel):
    # 获取图像和卷积核的尺寸
    img_h, img_w = image.shape
    k_h, k_w = kernel.shape

    # 计算padding大小
    pad_h = k_h // 2
    pad_w = k_w // 2

    # 对图像进行零填充
    padded_image = np.zeros((img_h + 2 * pad_h, img_w + 2 * pad_w))
    padded_image[pad_h:pad_h + img_h, pad_w:pad_w + img_w] = image

    # 初始化输出图像
    output = np.zeros((img_h, img_w))

    # 执行卷积操作
    for i in range(img_h):
        for j in range(img_w):
            # 提取当前窗口
            window = padded_image[i:i + k_h, j:j + k_w]
            # 计算卷积值
            output[i, j] = np.sum(window * kernel)

    return output
```

3.2.2 Sobel 边缘检测

实现步骤：

1. 将 RGB 图像转换为灰度图
2. 分别使用 Sobel X 和 Y 方向卷积核进行卷积
3. 计算梯度幅值并归一化

3.2.3 给定卷积核滤波

使用给定的卷积核 $\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$ （实际上就是 Sobel X 方向算子）对图像进行滤波。

3.2.4 颜色直方图计算

手动实现直方图统计，不调用现有函数库：

```
def calculate_histogram(image):  
    # 手动计算图像的颜色直方图  
    if len(image.shape) == 3:  
        # RGB图像  
        histograms = {}  
        channels = ['R', 'G', 'B']  
  
        for i, channel in enumerate(channels):  
            # 初始化直方图（256个bin）  
            hist = np.zeros(256, dtype=np.int32)  
  
            # 统计每个像素值的频次  
            channel_data = image[:, :, i].flatten()  
            for pixel_value in channel_data:  
                hist[pixel_value] += 1  
  
            histograms[channel] = hist  
    return histograms
```

3.2.5 GLCM 纹理特征提取

1. 将灰度图量化为 16 级
2. 在四个方向（ 0° 、 45° 、 90° 、 135° ）计算 GLCM
3. 从每个 GLCM 中提取 5 种纹理特征
4. 计算各方向特征的均值和标准差

3.3 实验流程

1. 加载自拍摄的输入图像
2. 执行 Sobel 算子边缘检测
3. 使用给定卷积核进行滤波
4. 计算并可视化 RGB 三通道颜色直方图
5. 提取 GLCM 纹理特征并保存为 NPY 格式
6. 生成结果对比图

四、实验结果

4.1 滤波结果对比

如图 1 所示，展示了原始图像、灰度图像、Sobel 边缘检测结果、给定卷积核滤波结果以及 X、Y 方向梯度图的对比效果。

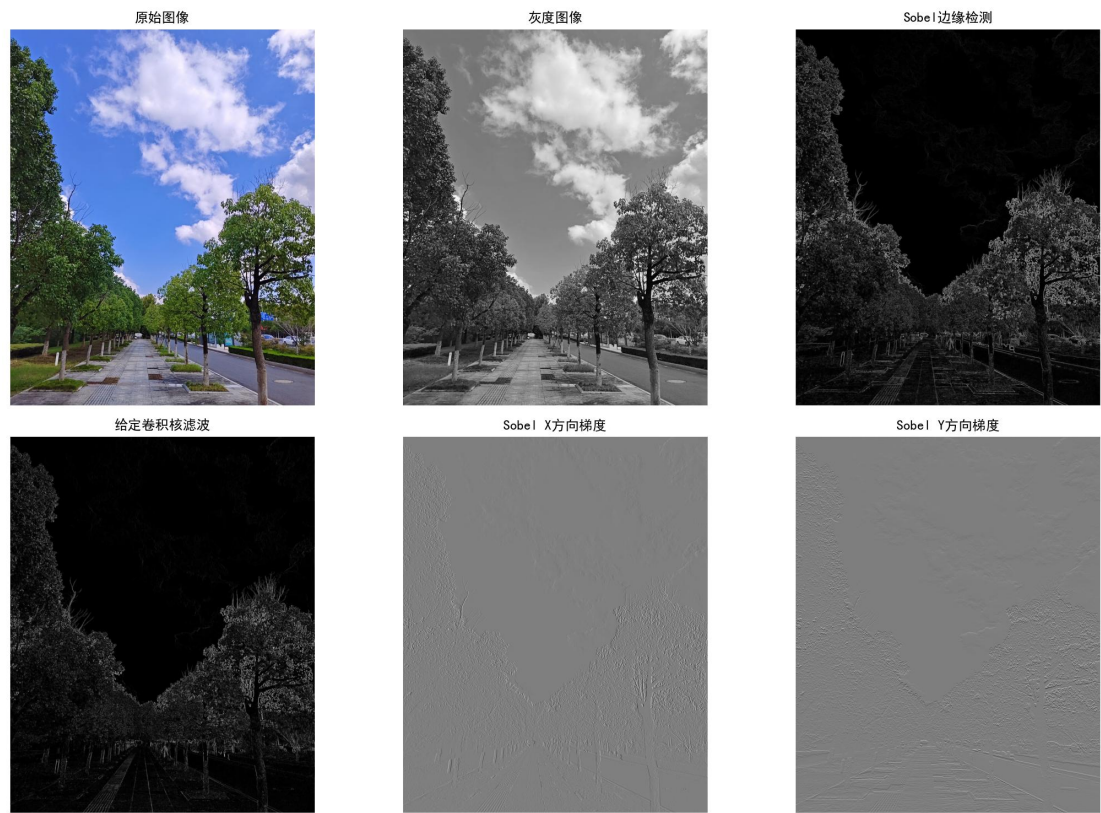


图 1：滤波结果对比图

从对比图中可以观察到：

- Sobel 边缘检测能够有效提取图像中的边缘信息
- X 方向梯度主要检测垂直边缘
- Y 方向梯度主要检测水平边缘
- 给定卷积核（Sobel X 算子）的滤波效果与 X 方向梯度一致

4.2 Sobel 边缘检测结果

如图 2 所示，Sobel 算子成功检测出了图像中的主要边缘特征，包括物体轮廓和纹理边界。



图 2: Sobel 边缘检测结果

4.3 给定卷积核滤波结果

如图 3 所示, 使用给定卷积核 $\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$ 对图像进行滤波后, 得到了图像在水平方向上的梯度信息。



图 3: 给定卷积核滤波结果

4.4 颜色直方图

如图 4 所示，分别展示了图像 R、G、B 三个通道的颜色直方图，直观地反映了图像中各颜色通道的像素分布情况。

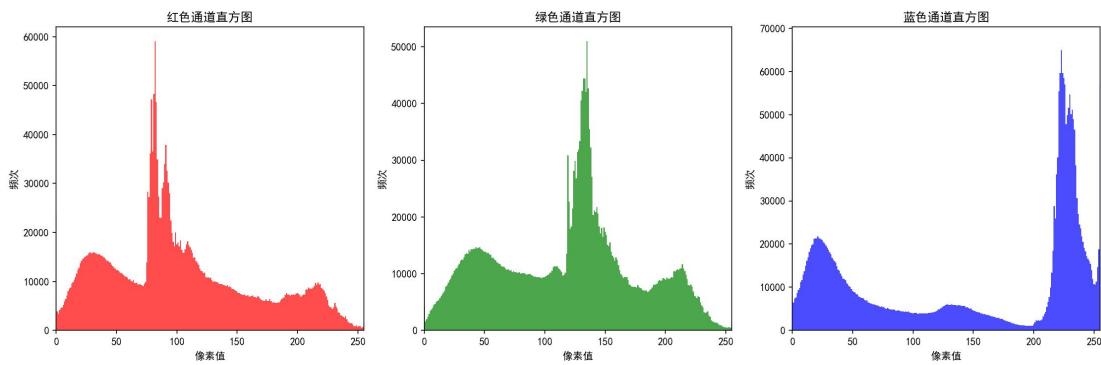


图 4：RGB 颜色直方图

R通道 - 最大频次：59039，总像素数：2893230
G通道 - 最大频次：50968，总像素数：2893230
B通道 - 最大频次：66975，总像素数：2893230

4.5 纹理特征

4.5.1 特征定义

提取的 GLCM 纹理特征已保存至 output/exp1/texture_features.npy，包含以下特征：

特征名称	数学公式	含义	取值范围
Contrast (对比度)	$\sum_{i,j} \ i - j\ ^2 p(i, j)$	衡量图像的清晰度和纹理沟纹深浅	$[0, +\infty)$
Correlation (相关性)	$\sum_{i,j} \frac{(i-\mu_i)(j-\mu_j)p(i,j)}{\sigma_i\sigma_j}$	衡量行列之间的灰度线性关系	$[-1, 1]$
Energy (能量)	$\sum_{i,j} p(i, j)^2$	衡量灰度分布的均匀程度	$[0, 1]$
Homogeneity (同质性)	$\sum_{i,j} \frac{p(i, j)}{1 + \ i - j\ }$	衡量图像纹理的局部均匀性	$[0, 1]$
Entropy (熵)	$-\sum_{i,j} p(i, j) \log_2 p(i, j)$	衡量图像的复杂度和随机性	$[0, \log_2(\text{levels}^2)]$

4.5.2 数值示例

图像的纹理特征提取结果如下：

```
提取纹理特征（GLCM）...
纹理特征：
- contrast: 2.497738
- contrast_std: 0.517431
- correlation: 0.912980
- correlation_std: 0.018038
- energy: 0.046139
- energy_std: 0.001955
- homogeneity: 0.782071
- homogeneity_std: 0.015837
- entropy: 5.543493
- entropy_std: 0.101045
```

五、实验体会

5.1 技术收获

- 深入理解卷积原理：通过手动实现 2D 卷积操作，深刻理解了卷积在图像处理中的作用机制，包括零填充、滑动窗口和加权求和等核心概念。
- 掌握边缘检测方法：Sobel 算子通过分别计算水平和垂直方向的梯度，能够有效检测图像中的边缘信息。不同方向的梯度图能够分别突出不同方向的边缘特征。
- 理解直方图统计意义：颜色直方图是图像最基本的统计特征之一，它不受图像旋转和平移的影响，是图像检索和匹配的重要特征。
- 学习纹理特征提取：GLCM 是一种经典的纹理描述方法，通过统计像素对的空间分布关系，能够有效描述图像的纹理特性。

5.2 实验反思

- 算法效率：手动实现的卷积算法采用双重循环，时间复杂度为 $O(n^2 \times k^2)$ ，其中 n 为图像尺寸， k 为卷积核尺寸。在实际应用中，可以使用向量化操作或 FFT 加速计算。
- 参数选择：GLCM 的参数（如距离 d 和角度 θ ）对结果有一定影响，需要根据具体应用场景进行调整。实验中采用 $d=1$ 和四个主要方向，适用于大多数纹理分析任务。
- 不使用函数包的意义：虽然增加了编程难度，但这种方式让我更加深入地理解了算法的内部工作原理，对今后使用高级 API（如 OpenCV、scikit-image）有更清晰的认识。
- 边界处理：在卷积操作中使用了零填充（zero padding）策略，确保输出图像与输入图像尺寸一致。其他策略还包括镜像填充、复制填充等。

5.3 性能分析

计算复杂度（注： n 为图像边长， k 为卷积核边长， c 为通道数， $levels$ 为灰度级数）：

操作	时间复杂度	空间复杂度	优化方法
2D卷积	$O(n^2k^2)$	$O(n^2)$	使用FFT变换，降至 $O(n^2\log n)$
Sobel边缘检测	$O(2n^2k^2)$	$O(3n^2)$	可并行计算X和Y方向
直方图统计	$O(n^2c)$	$O(256c)$	向量化操作
GLCM计算	$O(4n^2)$	$O(levels^2)$	使用稀疏矩阵

5.4 关键代码分析

5.4.1 零填充策略


```
# 计算padding大小
pad_h = k_h // 2
pad_w = k_w // 2

# 对图像进行零填充
padded_image = np.zeros((img_h + 2 * pad_h, img_w + 2 * pad_w))
padded_image[pad_h:pad_h + img_h, pad_w:pad_w + img_w] = image
```

这种填充方式确保了输出图像尺寸与输入一致，避免边界像素丢失。

5.4.2 GLCM 对称化

```
# 使GLCM对称
glcm = glcm + glcm.T
```

将 GLCM 与其转置相加，考虑了双向像素对关系，使统计更加鲁棒。

5.4.3 避免对数零错误

```
# 5. 熵 (Entropy): 衡量图像的随机性
# 避免log(0)
glcm_nonzero = glcm.copy()
glcm_nonzero[glcm_nonzero == 0] = 1e-10
features['entropy'] = -np.sum(glcm * np.log2(glcm_nonzero))
```

在计算熵时，避免 $\log(0)$ 导致的数值错误。

5.6 实验总结

本实验通过手动实现基础图像处理算法，我建立了对图像滤波、边缘检测和纹理分析的深刻理解。主要收获包括：

1. 理论联系实际：将数学公式转化为可执行代码
 2. 算法思维训练：理解算法的每一个细节和设计理由
 3. 工程实践能力：学会处理边界情况、数值稳定性等实际问题
 4. 为后续学习打基础：这些基础算法是理解深度学习卷积神经网络的基石
- 通过本实验，我不仅掌握了传统图像处理的核心技术，也为学习现代深度学习方法（如 CNN）奠定了坚实基础。